

# SDV\_Replica\_POC — Master Execution Plan

Tailored, step-wise, full-time solo schedule

**Author:** Chittaranjan Baral (@SJLNS)

**Version:** v1.0

**As of:** 2026-08-30 — repo tag v0.4.0

This supersedes the informal phase list from earlier — it's the single current source of truth, consolidating everything confirmed/fixed across v0.1.0–v0.4.0 into one forward plan. The two existing docs (SDV\_Replica\_POC\_Architecture\_Plan.md, SDV\_Replica\_POC\_Execution\_Guide.md) still hold the detailed \*how\* for each step — this document is the \*sequencing and pacing\* layer on top, tailored to working on this solo, full-time.

## 0. WHERE THINGS ACTUALLY STAND (READ THIS BEFORE ANYTHING ELSE)

| Category                | Status                                                                                                                                            |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Repo                    | Live, public, tagged v0.4.0 — <a href="https://github.com/SJLNS/SDV_Replica_POC">github.com/SJLNS/SDV_Replica_POC</a>                             |
| Real hardware confirmed | ZCU-1: STM32F407G-DISC1 (STM32F407VGT6) · ZCU-2: NUCLEO-F446RE (STM32F446RET6) · HPC-1: Raspberry Pi 5 16GB · HPC-2: BeagleBone Black Rev C       |
| ZCU firmware            | Skeleton compiles/links with real, correct memory maps (v0.4.0) — no real sensor/actuator logic yet                                               |
| HPC C++ services        | All 4 (hpc-bridge × 2, cloud-gateway × 2) compile/link/run as host sanity builds — no real gRPC logic yet, not cross-compiled for target arch yet |
| Build tooling           | scripts/buildenv.sh (Git Bash, ZCU) and scripts/build_console.bat (cmd.exe, all 6 modules) both proven working end-to-end on your actual laptop   |
| PKI                     | Script proven to work (root → intermediate → leaf, self-signature verified) — never run "for real" against actual boards yet                      |
| Physical hardware       | <b>Nothing has been touched yet</b> — no board has been booted, flashed, or wired outside this planning/tooling work                              |
| Ethernet                | Confirmed: neither ZCU board has onboard Ethernet — W5500 SPI module ×2 needed                                                                    |

**The honest headline: everything so far has been software/tooling groundwork.** That groundwork was worth doing properly (it's already caught and fixed five real bugs before they could waste bench time), but the actual POC — sensors, boards booting, signals flowing — starts now.

## 1. HOW THIS PLAN IS PACED

You're full-time on this alone, so the plan is a **day-by-day sequence**, not loose week buckets. A few pacing principles baked in:

- **Parallelizable work is flagged explicitly** — e.g. while Xen is compiling or a Yocto build is running unattended for hours, that's real time to do ZCU bench work instead of waiting.
- **The highest-risk item (Xen on RPi5) is front-loaded**, not saved for last — if it's going to blow the schedule, you want to know in week 1, not week 5.
- **Every day ends with a stated exit criterion** — a concrete, checkable thing that's either true or not, not a vague "make progress on X."
- Durations are **working-day estimates for one full-time person**, not optimistic best-case numbers — padding is already built in in the two places that have historically eaten time on this project (toolchain/environment setup, and anything Windows-shell-related).

## 2. DAY-BY-DAY PLAN

### Days 1 — Order the BOM, finish hardware validation prep

- Place the consolidated BOM order (W5500 ×2, multimeter, breadboard/jumpers/ resistor kit, USB-serial adapters ×2, sensor/actuator starter parts, RPi5 cooler + PSU if not already the official ones, 32GB+ microSD, small Ethernet switch, 5× Ethernet cables) — Execution Guide §0. This is a "start the shipping clock" action, not a blocking dependency for the next several days.
- Confirm Nucleo's onboard ST-Link and Discovery's onboard ST-Link/V2-A are both recognized by openocd/STM32CubeProgrammer from your laptop (Execution Guide §3.1 step 1–2). This needs zero new parts — do it today.

**Exit criterion:** BOM ordered; both ST-Link debuggers detected from your laptop.

### Day 2 — ZCU-1 standalone bring-up

- Flash a bare "blink an LED" project to the STM32F407G-DISC1 (has onboard LEDs — use those, no wiring needed yet).
- Confirm onboard user button reads correctly.
- This is the first real proof the whole toolchain → flash → run loop works on actual silicon, not just compiles.

**Exit criterion:** LED blinks under your own compiled firmware, button press detected.

### Day 3 — ZCU-2 standalone bring-up

- Same as Day 2, on the NUCLEO-F446RE (has an onboard user LED + button too).

**Exit criterion:** Same as Day 2, on the second board.

### Day 4 — RPi5 Stage 1 (stock OS validation)

- Flash Raspberry Pi OS 64-bit, boot, SSH in.
- `uname -m` → confirm aarch64.
- `vcgencmd measure_temp` idle, then under `stress-ng --cpu 4` load — record both numbers as your baseline before Xen/QNX load gets added later.
- Confirm Ethernet + DHCP lease.

**Exit criterion:** All four checks pass cleanly (Execution Guide §3.2).

### Day 5 — BBB Stage 1 (stock OS validation)

- Boot stock Debian (eMMC or fresh SD flash).
- Confirm serial console via USB-FTDI adapter — do this now, not during a future failed-boot panic.
- Confirm Ethernet + DHCP.

- Check `df -h` — note current eMMC headroom (the "512MB RAM / tight storage" risk starts being real from here on).

**Exit criterion:** All four checks pass cleanly (Execution Guide §3.3).

\*By end of Day 5: all 4 boards individually proven alive. This is the point past which "is it the board or is it my code" stops being an open question.\*

---

## Days 6–10 — RPi5 Xen bring-up (the highest-risk block — budget the full week)

This is the one place in the whole plan explicitly allowed to run long. Don't compress it.

- **Day 6:** Attempt the apt-package route (`xen-hypervisor-4.17-arm64`). Reboot, check `xl list`. If it boots clean → skip to Day 7. If boot config doesn't pick up the Xen kernel automatically, spend at most half a day hand-patching before falling back.
- **Day 7 (if needed):** Fall back to the xen-troops community RPi5 image. This is genuinely the more battle-tested path for this exact board — don't treat falling back as a failure, treat it as following the plan.
- **Day 8:** Once Dom0 is confirmed (`xl info` shows full CPU count), obtain the QNX SDP non-commercial license and **check the BSP explicitly lists RPi5 support before downloading anything larger** — this is the single biggest unknown left in the entire project.
- **Day 9–10:** Build the QNX DomU IFS image, create the Xen guest config (1 vCPU Dom0 / 3 vCPU DomU split), boot it, confirm `xl console qnx-domu` reaches a QNX prompt.

**Parallel work while Xen/Yocto builds run unattended:** ZCU real GPIO/ADC/PWM driver work (see Days 11+) can start here if Xen is taking multiple days — don't sit idle watching a build log.

**Exit criterion:** `xl list` shows both Dom0 and a running QNX DomU.

**If QNX BSP support turns out NOT to exist for RPi5:** stop, don't burn more days on it — fall back to running Yocto Linux in the DomU instead of QNX (same Xen partitioning story, different guest OS) and note the scope change in the architecture doc. This is a legitimate, planned fallback, not a failure state.

---

## Days 11–13 — BBB Yocto + Docker

Can start in parallel with the Xen block above if you want to keep two things moving — the Yocto build itself takes hours of unattended machine time.

- **Day 11:** Set up the Yocto build environment, add layers (`meta-ti`, `meta-openembedded`, `meta-virtualization`), configure for `beaglebone-yocto`, kick off `bitbake core-image-minimal`. This runs for hours — use the wait time for ZCU work.
- **Day 12:** Flash the resulting image, confirm boot, confirm `docker run hello-world` works on-device.
- **Day 13:** Buffer day for Yocto build failures (dependency issues, layer compatibility) — Yocto builds are notorious for needing a second pass.

**Exit criterion:** BBB boots Yocto, Docker runs a test container successfully.

---

## Day 14 — PKI, for real this time

- Generate the actual root CA + 3 intermediate CAs (not a throwaway test — this is the PKI the whole project runs on going forward).
- Issue real leaf certs to all 4 boards.
- Prove a bare `openssl s_server/s_client` mTLS handshake between HPC-1 and HPC-2 over the real Ethernet backbone, including a negative test (wrong cert correctly rejected).

**Exit criterion:** mTLS handshake succeeds on the real link; rejection test passes.

---

## Days 15–17 — ZCU real sensor/actuator I/O

- **Day 15:** Wire whatever sensors/actuators arrived from the BOM order (or whatever you already have) to ZCU-1. Bench-test each with a multimeter *\*before\** connecting to the board — confirm 3.3V logic compatibility.
- **Day 16:** Write real GPIO/ADC/PWM driver code replacing the placeholder loop in `zcu1-discovery/src/main.c`. Confirm real sensor values read correctly over serial console before adding networking.
- **Day 17:** Repeat Day 15–16 for ZCU-2.

**Exit criterion:** Both ZCUs read real sensor data and can drive a real actuator, confirmed over serial console (no networking yet).

---

## Days 18–19 — KUKSA + VSS, real deployment

- **Day 18:** Finalize `vss/vss_custom.vspec` against your actual wired sensors (not the placeholder example signals). Deploy KUKSA Databroker via Docker on both HPCs, pointed at real leaf certs from Day 14.
- **Day 19:** Smoke-test with `kuksa-client`: set a value, subscribe, confirm round-trip on both HPCs independently.

**Exit criterion:** Both HPCs run a databroker with your real VSS tree, reachable over TLS.

---

## Days 20–21 — W5500 wiring + MQTT feeder integration

*\*(W5500 modules should have arrived by now if ordered on Day 1 — if not, this is the point where a shipping delay actually starts costing you time; everything before this didn't need them.)\**

- **Day 20:** Wire W5500 to both ZCU boards, bring up the SPI-Ethernet driver, confirm each ZCU gets a real IP on the switch.
- **Day 21:** Add MQTT client + mbedTLS to ZCU firmware, write the MQTT-to-KUKSA feeder process on each HPC, prove one real sensor value travels ZCU → MQTT → HPC → VSS tree end to end.

**Exit criterion:** A live sensor reading on ZCU-1 appears in HPC-1's VSS tree within a reasonable latency; an actuation command set via `kuksa-client` reaches a real ZCU-1 relay/motor.

---

## Days 22–24 — HPC-to-HPC gRPC bridge, real logic

- Replace the `TODO` placeholders in all 4 `main.cpp` files with real gRPC/protobuf logic against `proto/hpc_bridge.proto`.
- Wire in mTLS 1.3 using the real HPC leaf certs (replacing `InsecureServerCredentials`).
- Implement the VSS subtree filter — this is the actual HPC-level filtering boundary the whole architecture doc describes.
- Test both directions: a ZCU-1 signal, filtered, becomes visible on HPC-2.

**Exit criterion:** A signal from ZCU-1 is observable (correctly filtered) on HPC-2's VSS tree, and vice versa — proving "any sensor to any cloud" up to the cloud boundary.

---

## Days 25–29 — Cloud integration (both branches)

- **Days 25–26:** AWS IoT Core — register your own `cloud-ca`, provision the HPC-1 Thing, issue+register the device cert, fill in real logic in `cloud-gateway's main.cpp`.
- **Days 27–28:** Same for Azure IoT Hub / HPC-2.

- **Day 29:** End-to-end proof both directions: a ZCU sensor value visible in a cloud dashboard/log, and a cloud-issued command reaching a real ZCU actuator, for both AWS and Azure branches independently.
- Exit criterion:** Both cloud branches prove sensor-to-cloud and cloud-to-actuator, independently.
- 

**Days 30–33 — OTA (SOTA)**

- HPC-1: Docker image versioning + rollback tag kept; QNX DomU image versioning.
  - HPC-2: Mender or swupdate for A/B rootfs updates.
  - Prove: push a new container image tag to a running HPC, confirm it deploys without a manual reflash, confirm rollback actually works.
- Exit criterion:** A real OTA push-and-rollback cycle demonstrated on both HPCs.
- 

**Day 34+ — Stretch features (open-ended, deliberately last)**

In priority order (per the original architecture doc's MoSCoW list): 1. Predictive analytics on the QNX DomU (or Yocto DomU fallback) 2. Digital Twin (cloud-side mirrored state + simple dashboard) 3. Digital Key (cloud-issued short-lived token, HPC-side validator) 4. Voice UI (thin client → cloud intent API) 5. ZCU FOTA (separate bootloader sub-project — genuinely its own effort) 6. SOME/IP as an HPC-HPC service-discovery exercise, if you want the extra learning

**No exit criterion for this block** — it's explicitly open-ended, scope it to your remaining energy/interest once the core system (through Day 33) works.

---

**3. FULL-TIME TOTAL ESTIMATE**

| Block                                             | Days                     | Calendar (working days)     |
|---------------------------------------------------|--------------------------|-----------------------------|
| HW validation + prep (§Days 1–5)                  | 5                        | Week 1                      |
| RPi5 Xen (§Days 6–10)                             | 5 (highest overrun risk) | Week 2                      |
| BBB Yocto+Docker (§Days 11–13, parallel-eligible) | 3                        | overlaps Week 2             |
| PKI real (§Day 14)                                | 1                        | Week 3 start                |
| ZCU real I/O (§Days 15–17)                        | 3                        | Week 3                      |
| KUKSA+VSS (§Days 18–19)                           | 2                        | Week 3                      |
| MQTT feeder + W5500 (§Days 20–21)                 | 2                        | Week 4                      |
| HPC bridge real logic (§Days 22–24)               | 3                        | Week 4                      |
| Cloud integration (§Days 25–29)                   | 5                        | Week 5                      |
| OTA (§Days 30–33)                                 | 4                        | Week 6 (partial)            |
| <b>Core system total</b>                          | <b>~33 working days</b>  | <b>~6.5 weeks full-time</b> |
| Stretch features                                  | open-ended               | beyond                      |

This is a real estimate, not a rounded-down best case — it already assumes the Xen week runs its full length and one buffer day for Yocto. If Xen goes faster than budgeted, that time flows straight into starting ZCU work early rather than sitting unused.

---

## 4. WHAT COULD STILL PUSH THIS OUT FURTHER

Carried forward from the risk register, still open:

| Risk                                                    | Where it hits | Mitigation already in this plan                                                                              |
|---------------------------------------------------------|---------------|--------------------------------------------------------------------------------------------------------------|
| QNX BSP has no confirmed RPi5 support                   | Days 8–10     | Explicit fallback to Yocto-in-DomU instead of QNX, decided by Day 8, not discovered on Day 10                |
| Yocto build dependency hell                             | Days 11–13    | Day 13 is an explicit buffer, not optimism                                                                   |
| W5500 shipping delay                                    | Day 20        | Ordered Day 1, ~3 weeks of runway before it's actually needed                                                |
| KUKSA Databroker mTLS support unclear at pinned version | Day 18        | Verify before building the security story on it; sidecar fallback already documented in the architecture doc |

## 5. HOW TO USE THIS DOCUMENT DAY TO DAY

- Each day's exit criterion is your "did today actually work" check — if you hit it, move to the next day; if not, that's your natural place to stop and either debug or ask for help before compounding the delay onto the next step.
- When a day's work surfaces a real bug or decision (like the STM32 part numbers or the CRLF issue did), that's worth its own commit + CHANGELOG entry + tag, same pattern as `v0.1.0` through `v0.4.0` — keep doing that, it's been genuinely useful for tracking what actually happened vs. what was planned.
- Report back at the end of each block (not necessarily every single day) with what actually happened — the plan above is a good-faith estimate, not a commitment, and it should flex based on what Days 1–5 actually show about your pace.